

深圳大学实验报告

课程名称： 计算机系统(2)

实验项目名称： 逆向工程实验

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 马晨琳

报告人： 黄嘉聪 学号： 2023192044 班级： 1

实验时间： 2025 年 4 月 16 日 ~ 2024 年 4 月 29 日

实验报告提交时间： 2024 年 4 月 29 日

一、实验目的：

1. 理解程序（控制语句、函数、返回值、堆栈结构）是如何运行的
2. 掌握 GDB 调试工具和 objdump 反汇编工具

二、实验环境：

1. 计算机（Intel CPU）
2. Linux64 位操作系统
3. GDB 调试工具
4. objdump 反汇编工具

三、实验内容

本实验设计为一个黑客拆解二进制炸弹的游戏。我们仅给黑客（同学）提供一个二进制可执行文件 bomb 和主函数所在的源程序 bomb.c，不提供每个关卡的源代码。程序运行中有 6 个关卡（6 个 phase），每个关卡需要用户输入正确的字符串或数字才能通关，否则会引爆炸弹（打印出一条错误信息，并导致评分下降）！

要求同学运用 **GDB 调试工具**和 **objdump 反汇编工具**，通过分析汇编代码，找到在每个 phase 程序段中，引导程序跳转到“explode_bomb”程序段的地方，并分析其成功跳转的条件，以此为突破口寻找应该在命令行输入何种字符串来通关。

本实验要求解决 Phase_1、Phase_2、Phase_3、Phase_4、Phase_5、Phase_6。通过截图把结果写在实验报告上。

四、实验步骤和结果

1. 代码整体结构分析

①如图 1 所示，首先在终端里面使用【cd】命令进入到存储代码的对应文件夹中

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~$ cd experiment3
```

图 1 进入文件夹指令

然后如图 2 所示输入反汇编命令【objdump -d bomb > 1.txt】保存汇编代码至 1.txt 文件中

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/experiment3$ objdump -d bomb_64.dat > 1.txt
```

图 2 保存反汇编文件指令

如图三所示，在文件夹里面出现汇编代码 1.txt。

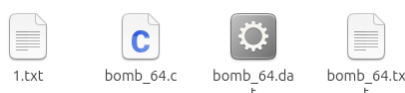


图 3 文件夹中的汇编代码文件 1.txt

②如图 4 所示，阅读汇编代码的 main 函数部分，发现调用了从 phase1 到 phase6 这六个函数，说明该六个函数分别对应六个观看，故接下来逐个对其进行分析。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

288	400db4: e8 b7 00 00 00	call 400e70 <phase_1>
289	400db9: e8 c7 09 00 00	call 401785 <phase_defused>
290	400dbe: bf e8 19 40 00	mov \$0x4019e8,%edi
291	400dc3: e8 d8 fc ff ff	call 400aa0 <puts@plt>
292	400dc8: e8 92 08 00 00	call 40165f <read_line>
293	400dcd: 48 89 c7	mov %rax,%rdi
294	400dd0: e8 b7 00 00 00	call 400e8c <phase_2>
295	400dd5: e8 ab 09 00 00	call 401785 <phase_defused>
296	400dda: bf 47 19 40 00	mov \$0x401947,%edi
297	400ddf: e8 bc fc ff ff	call 400aa0 <puts@plt>
298	400de4: e8 76 08 00 00	call 40165f <read_line>
299	400de9: 48 89 c7	mov %rax,%rdi
300	400dec: e8 08 01 00 00	call 400ef9 <phase_3>
301	400df1: e8 8f 09 00 00	call 401785 <phase_defused>
302	400df6: bf 65 19 40 00	mov \$0x401965,%edi
303	400dfb: e8 a0 fc ff ff	call 400aa0 <puts@plt>
304	400e00: e8 5a 08 00 00	call 40165f <read_line>
305	400e05: 48 89 c7	mov %rax,%rdi
306	400e08: e8 b4 01 00 00	call 400fc1 <phase_4>
307	400e0d: e8 73 09 00 00	call 401785 <phase_defused>
308	400e12: bf 18 1a 40 00	mov \$0x401a18,%edi
309	400e17: e8 84 fc ff ff	call 400aa0 <puts@plt>
310	400e1c: e8 3e 08 00 00	call 40165f <read_line>
311	400e21: 48 89 c7	mov %rax,%rdi
312	400e24: e8 d9 01 00 00	call 401002 <phase_5>
313	400e29: e8 57 09 00 00	call 401785 <phase_defused>
314	400e2e: bf 40 1a 40 00	mov \$0x401a40,%edi
315	400e33: e8 68 fc ff ff	call 400aa0 <puts@plt>
316	400e38: bf 78 1a 40 00	mov \$0x401a78,%edi
317	400e3d: e8 5e fc ff ff	call 400aa0 <puts@plt>
318	400e42: bf b8 1a 40 00	mov \$0x401ab8,%edi
319	400e47: e8 54 fc ff ff	call 400aa0 <puts@plt>
320	400e4c: e8 0e 08 00 00	call 40165f <read_line>
321	400e51: 48 89 c7	mov %rax,%rdi
322	400e54: e8 80 02 00 00	call 4010d9 <phase_6>
323	400e59: e8 27 09 00 00	call 401785 <phase_defused>

图 4 汇编代码中 main 函数选段

③同时，可以获知主函数当中调用的各个`phase`函数前的三条指令的作用，即读取程序输入的内容，并作为各个`phase`函数的参数传入到各个函数中。

2. phase_1|关卡①

如图 5 所示，即为 `phase_1` 的代码内容：

339	400e70 <phase_1>:	
340	48 83 ec 08	sub \$0x8,%rsp # 函数初始化
341		
342	be f8 1a 40 00	mov \$0x401af8,%esi # %esi(传入参数)=0x401af8
343	e8 bf 03 00 00	call 40123d <strings_not_equal> # 调用函数strings_not_equal
344		
345	85 c0	test %eax,%eax
346	74 05	je 400e87 <phase_1+0x17> # 判断返回值是否为0
347	e8 b6 07 00 00	call 40163d <explode_bomb> # 如果不为0，引爆“炸弹”
348		
349	48 83 c4 08	add \$0x8,%rsp # 函数结束并回收栈空间
350	c3	ret

图 5 汇编代码中 `phase_1` 函数

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

代码主要逻辑分析：

- ①除去前后两段的函数初始化部分，342行所对应的操作为将作为地址的立即数【0x401af8】传入寄存器【%esi】中，然后调用函数< strings_not_equal >进行判断
- ②在判断完成后，传回的返回参数存储在寄存器【%eax】当中，然后通过 345 行的【test】指令与【je】指令判断其是否为0，然后进行跳转，如果为 0，即字符串相同则跳转到349行的函数结尾处。
- ③反之，则由程序运行的顺序运行到347行处调用<explode_bomb>，引爆炸弹。
- ④如图 6 所示即函数< strings_not_equal >的内容，可以看出若字符串完全相同，则其返回值寄存器【%eax】会赋值为0

```
658 00000000040123d <strings_not_equal>:
659 40123d: 48 89 5c 24 e8      mov     %rbx, -0x18(%rsp)
660 401242: 48 89 6c 24 f0      mov     %rbp, -0x10(%rsp)
661 401247: 4c 89 64 24 f8      mov     %r12, -0x8(%rsp)
662 40124c: 48 83 ec 18         sub     $0x18, %rsp # 分配函数栈帧
663
664 401250: 48 89 fb           mov     %rdi, %rbx
665 401253: 48 89 f5           mov     %rsi, %rbp
666 401256: e8 c6 ff ff ff     call    401221 <string_length> # 判断字符串长度是否相等
667 40125b: 41 89 c4           mov     %eax, %r12d
668 40125e: 48 89 ef           mov     %rbp, %rdi
669 401261: e8 bb ff ff ff     call    401221 <string_length> # 判断字符串长度是否相等
670 401266: ba 01 00 00 00     mov     $0x1, %edx
671 40126b: 41 39 c4           cmp     %eax, %r12d
672 40126e: 75 36             jne     4012a6 <strings_not_equal+0x69> # 判断字符串是否相等，相等就赋值为0
673 401270: 0f b6 03          movzbl  (%rbx), %eax
674 401273: b2 00            mov     %eax, %dl
675 401275: 84 c0            test    %al, %al
676 401277: 74 2d            je      4012a6 <strings_not_equal+0x69> # 判断字符串是否相等，相等就赋值为0
677 401279: b2 01            mov     $0x1, %dl
678 40127b: 3a 45 00         cmp     0x0(%rbp), %al
679 40127e: 75 26            jne     4012a6 <strings_not_equal+0x69> # 判断字符串是否相等，相等就赋值为0
680 401280: b8 00 00 00 00     mov     $0x0, %eax
681 401285: eb 0a            jmp     401291 <strings_not_equal+0x54> # 判断字符串是否相等，相等就赋值为0
682 401287: 48 83 c0 01       add     $0x1, %rax
683 40128b: 3a 54 05 00       cmp     0x0(%rbp, %rax, 1), %dl
684 40128f: 75 10            jne     4012a1 <strings_not_equal+0x64> # 判断字符串是否相等，相等就赋值为0
685 401291: 0f b6 54 03 01     movzbl 0x1(%rbx, %rax, 1), %edx
686 401296: 84 d2            test    %dl, %dl
687 401298: 75 ed            jne     401287 <strings_not_equal+0x4a> # 判断字符串是否相等，相等就赋值为0
688 40129a: ba 00 00 00 00     mov     $0x0, %edx
689 40129f: eb 05            jmp     4012a6 <strings_not_equal+0x69> # 判断都成功就不对返回值进行处理（即为0）
690
691 4012a1: ba 01 00 00 00     mov     $0x1, %edx # 字符串不相等，赋值为1
692 4012a6: 89 d0            mov     %edx, %eax
```

图 6 汇编代码中 strings_not_equal 函数

输入内容获取：

综上所述，需要输入与M[0x401af8]相同的字符串。

- ①利用 gdb 调试工具来破解 Phase1 中的神秘字符串。如图 7 所示，输入指令【gdb bomb_64.dat】对程序 bomb_64.dat 进行调试。

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/experiment3$ gdb bomb_64.dat
GNU gdb (Ubuntu 16.2-8ubuntu1) 16.2
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
```

图 7 启用 gdb 调试

- ②启用调试后，如图 8 使用指令【p (char*)0x401af8】以 char* 字符数组的形式读取存储在地址0x401af8的内容。结果如图 8 所示为：Science isn't about why, it's about why not?

```
(gdb) p (char*)0x401af8
$1 = 0x401af8 "Science isn't about why, it's about why not?"
```

图 8 gdb 读取对应地址内容

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

③使用指令【r】对程序启用运行调试，如图 9 输入该字符串，由提示可知答案正确。

```
(gdb) r
Starting program: /home/huangjiacong_2023192044/experiment3/bomb_64.dat
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Science isn't about why, it's about why not?
Phase 1 defused. How about the next one?
```

图 9 gdb 测试第一部分对应答案

3. phase_2|关卡②

如图 10 所示，即为 phase_2 所对应代码：

```
352 000000000400e8c <phase_2>:
353 400e8c: 48 89 5c 24 e0      mov     %rbx,-0x20(%rsp) #rbx = 0
354 400e91: 48 89 6c 24 e8      mov     %rbp,-0x18(%rsp) #rbp = 0
355 400e96: 4c 89 64 24 f0      mov     %r12,-0x10(%rsp) #r12 = 0
356 400e9b: 4c 89 6c 24 f8      mov     %r13,-0x8(%rsp) #r13 = 0
357 400ea0: 48 83 ec 48         sub     $0x48,%rsp #rsp -= 0x48 # 分配函数栈帧
358
359 400ea4: 48 89 e6           mov     %rsp,%rsi # rsi = rsp
360 400ea7: e8 97 08 00 00     call    401743 <read_six_numbers>
361 400eac: 48 89 e5           mov     %rsp,%rbp # rbp = rsp
362 400eaf: 4c 8d 6c 24 0c     lea     0xc(%rsp),%r13 # r13 = rsp + 0xc
363 400eb4: 41 bc 00 00 00 00  mov     $0x0,%r12d # r12 = 0
364 400eba: 48 89 eb           mov     %rbp,%rbx # rbx = rbp
365 400ebd: 8b 45 0c           mov     0xc(%rbp),%eax # eax = *(rbp + 0xc)
366 400ec0: 39 45 00           cmp     %eax,0x0(%rbp) # cmp *(rbp + 0xc), *(rbp + 0x0)
367 400ec3: 74 05             je      400eca <phase_2+0x3e> # if *(rbp + 0xc) == *(rbp + 0x0) goto 400eca
368 400ec5: e8 73 07 00 00     call    40163d <explode_bomb> # 如果不相等，那就引爆炸弹
369 400eca: 44 03 23           add     (%rbx),%r12d # r12 += *(rbx)，即统计数字综合
370 400ecd: 48 83 c5 04       add     $0x4,%rbp # rbp += 0x4，即更换下一个数字
371 400ee1: 4c 39 ed           cmp     %r13,%rbp # cmp r13, rbp
372 400ed4: 75 e4             jne     400eba <phase_2+0x2e> # if r13 != rbp goto 400eba
373 400ed6: 45 85 e4          test    %r12d,%r12d
374 400ed9: 75 05             jne     400ee0 <phase_2+0x54> # if r12 <= 0 goto 400ee0
375 400edb: e8 5d 07 00 00     call    40163d <explode_bomb>
376
377 400ee0: 48 8b 5c 24 28     mov     0x28(%rsp),%rbx # 回收栈帧
378 400ee5: 48 8b 6c 24 30     mov     0x30(%rsp),%rbp
379 400eea: 4c 8b 64 24 38     mov     0x38(%rsp),%r12
380 400eef: 4c 8b 6c 24 40     mov     0x40(%rsp),%r13
381 400ef4: 48 83 c4 48       add     $0x48,%rsp
382 400ef8: c3               ret
```

图 10 汇编代码中 phase_2 函数

代码主要逻辑分析：

①除去初始化部分，从359行开始函数内容。首先将当前的栈指针【%rsp】作为函数参数传入到引用的函数<read_six_numbers>当中。然后将读取数字完毕后的栈顶指针传入到【%rbp】中进行保存。

②初始化。将【%r12】赋值为0，将【%r13】赋值为对应的第四个数字（即当前数字+0xc（移动地址距离）/4（int 类型数据大小）= 3，也即移动三个数字到第四个数字。即【%r13 = M[%rbp + 0xc]】

③通过初始化后，函数逐个对 M[%rbp]和 M[%rbp+0xc]进行判断，直到超出范围判断结束，即逐个判断输入的第一、二、三位和第四、五、六位的数字是否相等。若不相等则引爆炸弹。

④在过程当中对前三位数字进行求和，若为0则引爆炸弹。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

输入内容获取：

综上所述，需要第一二三位的数字与第四五六位的数字分别相等且总和不为0，故如图 11、图 12 所示可取【1204 1204 1204 1204 1204】或【1 2 3 1 2 3】等作为输入，输入后由提示可知答案正确。

```
Phase 1 defused. How about the next one?
1204 1204 1204 1204 1204 1204
```

图 11 gdb 测试第二部分对应答案①

```
Phase 1 defused. How about the next one?
1 2 3 1 2 3
That's number 2. Keep going!
```

图 12 gdb 测试第二部分对应答案②

4. phase_3|关卡③

如图 13 所示，即为 phase_3 所对应代码：

```
384 00000000400ef9 <phase_3>:
385 400ef9: 48 83 ec 18      sub    $0x18,%rsp
386
387 400efd: 48 8d 4c 24 08    lea    0x8(%rsp),%rcx #rcx = rsp + 0x8
388 400f02: 48 8d 54 24 0c    lea    0xc(%rsp),%rdx #rdx = rsp + 0xc
389 400f07: be be 1e 40 00    mov    $0x401ebe,%esi #esi = 0x401ebe
390 400f0c: b8 00 00 00 00    mov    $0x0,%eax #eax = 0
391 400f11: e8 9a fb ff ff    call   400ab0 <__isoc99_sscanf@plt> # scanf("%d %d", rsp + 0xc, rsp + 0x8 )
392 400f16: 83 f8 01          cmp    $0x1,%eax # cmp eax, 0x1
393 400f19: 7f 05            jg     400f20 <phase_3+0x27>
394 400f1b: e8 1d 07 00 00    call   40163d <explode_bomb>
395
396 400f20: 83 7c 24 0c 07    cmpl   $0x7,0xc(%rsp) # cmp *(rsp + 0xc), 0x7
397 400f25: 77 3c            ja     400f63 <phase_3+0x6a> # if *(rsp + 0xc) > 0x7 goto bomb,即超出case范围
398 400f27: b8 44 24 0c      mov    0xc(%rsp),%eax # eax = *(rsp + 0xc)
399 400f2b: ff 24 c5 60 1b 40 00 # jmp *(0x401b60(,%rax,8) # jmp *(0x401b60 + 8 * eax),即跳转到对应的case
400 400f32: b8 17 02 00 00    mov    $0x217,%eax # case 0
401 400f37: eb 3b            jmp    400f74 <phase_3+0x7b>
402 400f39: b8 d6 00 00 00    mov    $0xd6,%eax # case 2
403 400f3e: eb 34            jmp    400f74 <phase_3+0x7b>
404 400f40: b8 53 01 00 00    mov    $0x153,%eax # case 3
405 400f45: eb 2d            jmp    400f74 <phase_3+0x7b>
406 400f47: b8 77 00 00 00    mov    $0x77,%eax # case 4
407 400f4c: eb 26            jmp    400f74 <phase_3+0x7b>
408 400f4e: b8 60 01 00 00    mov    $0x160,%eax # case 5
409 400f53: eb 1f            jmp    400f74 <phase_3+0x7b>
410 400f55: b8 97 03 00 00    mov    $0x397,%eax # case 6
411 400f5a: eb 18            jmp    400f74 <phase_3+0x7b>
412 400f5c: b8 9c 01 00 00    mov    $0x19c,%eax # case 7
413 400f61: eb 11            jmp    400f74 <phase_3+0x7b>
414 400f63: e8 d5 06 00 00    call   40163d <explode_bomb>
415 400f68: b8 00 00 00 00    mov    $0x0,%eax
416 400f6d: eb 05            jmp    400f74 <phase_3+0x7b>
417 400f6f: b8 9e 03 00 00    mov    $0x39e,%eax # case 1
418
419 400f74: 3b 44 24 08      cmp    0x8(%rsp),%eax # cmp *(rsp + 0x8), eax
420 400f78: 74 05            je     400f7f <phase_3+0x86> # if *(rsp + 0x8) == eax goto end
421 400f7a: e8 be 06 00 00    call   40163d <explode_bomb>
422
423 400f7f: 48 83 c4 18      add    $0x18,%rsp
424 400f83: c3              ret
```

图 13 汇编代码中 phase_3 函数

代码主要逻辑分析：

①除去初始化，即进行数字的输入判断，也即对应代码，通过主函数可知，其对应 C 中的输入函数 `scanf("%d %d", rsp + 0xc, rsp + 0x8)`，其中 `rsp + 0xc`, `rsp + 0x8` 为输入的两个数字。同时进行判断，如果输入失败，也即该函数返回值 `[%eax]` 为 0，则直接引爆炸弹。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

②类似于 switch 函数，对输入的第一个数字进行判断，如果其大于 7，也即超过 switch 函数可以判断的范围，则直接引爆炸弹。若无，则根据输入的第一个数字进行对应的跳转，并赋值给【%eax】准备下一步的判断。

③将输入的第二个数字与【%eax】进行相等的判断，若失败则引爆炸弹，反之则通过。

输入内容获取：

综上所述，有如下的对应答案表：

第一个数字	0	1	2	3
[%eax]	0x217	0x39e	0xd6	0x153
第二个数字	535	926	214	339
第一个数字	4	5	6	7
[%eax]	0x77	0x160	0x397	0x19c
第二个数字	119	352	919	412

表 1 关卡③对应答案表

根据表 1，如图 14，输入其中的一个对应编号如【2 214】，可以看到提示，即答案正确。

```
That's number 2. Keep going!
2 214
Halfway there!
```

图 14 gdb 测试第三部分对应答案

5. phase_4|关卡④

如图 15 所示，即为 phase_4 所对应代码：

```

445  000000000400fc1 <phase_4>:
446      400fc1: 48 83 ec 18          sub     $0x18,%rsp
447
448      400fc5: 48 8d 54 24 0c       lea     0xc(%rsp),%rdx #rdx = rsp + 0xc
449      400fca: be c1 1e 40 00       mov     $0x401ec1,%esi #esi = 0x401ec1
450      400fcf: b8 00 00 00 00       mov     $0x0,%eax
451      400fd4: e8 d7 fa ff ff       call    400ab0 <__isoc99_sscanf@plt> # scanf("%d", rsp + 0xc)
452      400fd9: 83 f8 01             cmp     $0x1,%eax
453      400fdc: 75 07               jne     400fe5 <phase_4+0x24>
454
455      400fde: 83 7c 24 0c 00       cmpl    $0x0,0xc(%rsp) # cmp *(rsp + 0xc), 0x0
456      400fe3: 7f 05               jg      400fea <phase_4+0x29> # if *(rsp + 0xc) > 0
457      400fe5: e8 53 06 00 00       call    40163d <explode_bomb>
458      400fea: 8b 7c 24 0c          mov     0xc(%rsp),%edi # edi = *(rsp + 0xc)
459      400fee: e8 91 ff ff ff       call    400f84 <func4>
460      400ff3: 83 f8 37             cmp     $0x37,%eax # cmp eax, 0x37
461      400ff6: 74 05               je      400ffd <phase_4+0x3c> # if eax == 0x37 goto end
462      400ff8: e8 40 06 00 00       call    40163d <explode_bomb>
463      400ffd: 48 83 c4 18          add     $0x18,%rsp
464      401001: c3                  ret

```

图 15 汇编代码中 phase_4 函数

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

代码主要逻辑分析：

①除去初始化，即进行数字的输入判断，也即对应代码，通过主函数可知，其对应 C 中的输入函数 `scanf("%d", rsp + 0xc)`，其中 `rsp + 0xc` 为输入的数字。同时进行判断，如果输入失败，也即该函数返回值[%eax]为0，则直接引爆炸弹。

②判断输入的数字，如果小于等于零则直接引爆炸弹，反之则将输入的数字赋值给【%edi】传入函数<func4>中。

③如图 16 所示，即<func4>所对应代码：

```
426 00000000400f84 <func4>:
427 400f84: 48 89 5c 24 f0      mov     %rbx,-0x10(%rsp)
428 400f89: 48 89 6c 24 f8      mov     %rbp,-0x8(%rsp)
429 400f8e: 48 83 ec 18         sub     $0x18,%rsp
430
431 400f92: 89 fb              mov     %edi,%ebx # ebx = edi
432 400f94: b8 01 00 00 00     mov     $0x1,%eax # eax = 0x1, 赋初值后判断是否要继续
433 400f99: 83 ff 01          cmp     $0x1,%edi # cmp edi, 0x1
434 400f9c: 7e 14             jle     400fb2 <func4+0x2e> # if edi <= 1 goto end, 超出范围了就直接结束
435 400f9e: 8d 7b ff          lea     -0x1(%rbx),%edi # edi = ebx - 0x1
436 400fa1: e8 de ff ff ff    call    400f84 <func4> # func4(ebx - 0x1)
437 400fa6: 89 c5             mov     %eax,%ebp
438 400fa8: 8d 7b fe          lea     -0x2(%rbx),%edi # edi = ebx - 0x2
439 400fab: e8 d4 ff ff ff    call    400f84 <func4> # func4(ebx - 0x2)
440 400fb0: 01 e8            add     %ebp,%eax # eax += ebp
441 # 没超出范围就递归引用返回func4(ebx - 0x1) + func4(ebx - 0x2)
442
443 400fb2: 48 8b 5c 24 08     mov     0x8(%rsp),%rbx
444 400fb7: 48 8b 6c 24 10     mov     0x10(%rsp),%rbp
445 400fbc: 48 83 c4 18         add     $0x18,%rsp
446 400fc0: c3               ret
```

图 16 汇编代码中 func4 函数

可以获知，<func4>中的内容为：

1) 若传入参数[%edi]<=1，则返回1；

2) 若传入参数[%edi]>=1，则返回<func4([%edi]-1)>+<func4([%edi]-2)>；

即递归调用函数形成类似斐波那契数列的结构，而其传入的参数为其在斐波那契数列中的位置。

④获取到<func4>传回的返回值[%eax]后，将其与\$37（即 55）进行对比，若不等则引爆炸弹，反之结束函数，进入下一个部分。

输入内容获取：

综上所述，需要在将对应的参数传入<func4>后，其返回值要等于55。由斐波那契数列的性质，经过数学计算可知其对应斐波那契数列的第九位，故要输入进的答案参数为【9】。如图 17，输入答案【9】，可以看到对应提示，即答案正确。

```
Halfway there!
9
So you got that one. Try this one.
```

图 17 gdb 测试第四部分对应答案

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

6. phase_5|关卡⑤

如图 18 所示，即为 phase_5 所对应代码：

```
469 000000000401002 <phase_5>:
470 401002: 48 83 ec 18      sub    $0x18,%rsp
471
472 401006: 48 8d 4c 24 08    lea    0x8(%rsp),%rcx
473 40100b: 48 8d 54 24 0c    lea    0xc(%rsp),%rdx
474 401010: be be 1e 40 00    mov    $0x401ebe,%esi
475 401015: b8 00 00 00 00    mov    $0x0,%eax
476 40101a: e8 91 fa ff ff    call   400ab0 <__isoc99_sscanf@plt> # scanf("%d %d", rsp + 0x8, rsp + 0xc)
477 40101f: 83 f8 01          cmp    $0x1,%eax
478 401022: 7f 05            jg     401029 <phase_5+0x27>
479 401024: e8 14 06 00 00    call   40163d <explode_bomb>
480
481 401029: 8b 44 24 0c       mov    0xc(%rsp),%eax # eax = *(rsp + 0xc)
482 40102d: 83 e0 0f          and    $0xf,%eax      # 代码截断到0-15范围内
483 401030: 89 44 24 0c       mov    %eax,0xc(%rsp) # *(rsp + 0xc) &= 0xf=eax
484 401034: 83 f8 0f          cmp    $0xf,%eax
485 401037: 74 2c            je     401065 <phase_5+0x63> # 如果截断后数字等于15，则直接爆炸
486
487 401039: b9 00 00 00 00    mov    $0x0,%ecx #ecx = 0x0
488 40103e: ba 00 00 00 00    mov    $0x0,%edx #edx = 0x0
489
490 401043: 83 c2 01          add    $0x1,%edx # edx += 0x1
491 401046: 48 98            cltq
492 401048: 8b 04 85 a0 1b 40 00 mov    0x401ba0(,%rax,4),%eax # eax|rax = *(0x401ba0 + %rax*4)
493 40104f: 01 c1            add    %eax,%ecx      # ecx = eax+ecx
494 401051: 83 f8 0f          cmp    $0xf,%eax
495 401054: 75 ed            jne    401043 <phase_5+0x41> # if eax != 0xf goto 401043, 即继续递归下一个
496
497 401056: 89 44 24 0c       mov    %eax,0xc(%rsp) # *(rsp + 0xc) = eax
498 40105a: 83 fa 0c          cmp    $0xc,%edx #cmp edx, 0xc
499 40105d: 75 06            jne    401065 <phase_5+0x63> # if edx != 0xc goto bomb
500 40105f: 3b 4c 24 08       cmp    0x8(%rsp),%ecx #cmp *(rsp + 0x8), ecx
501 401063: 74 05            je     40106a <phase_5+0x68> #if ecx == *(rsp + 0x8) goto end
502 401065: e8 d3 05 00 00    call   40163d <explode_bomb>
503
504 40106a: 48 83 c4 18       add    $0x18,%rsp
505 40106e: c3              ret
```

图 18 汇编代码中 phase_5 函数

代码主要逻辑分析：

①除去函数初始化，即类似关卡③，其对应 C 中的输入函数 `scanf("%d %d", rsp + 0xc, rsp + 0x8)`，其中 `rsp + 0xc`, `rsp + 0x8` 为输入的两个数字。同时进行判断，如果输入失败，也即该函数返回值[%eax]为 0，则直接引爆炸弹。

②对输入的第二个数字[%rsp+0xc]进行截断，使之大小在 0-15 范围内，并判断是否等于 15，如果等于则直接引爆炸弹。

③通过输入的数字进行寻址，若寻址结果不等于 15 则使用地址所对应的数字基于 `0x401ba0` 进行偏移后再次寻址，至找到 15 后退出，在此过程中使用寄存器【%edx】记录递归寻址的次数，使用寄存器【%ecx】记录过程当中经过所有数字的和。

④最后判断寻址的次数是否等于 `0xc`（12），与经过数字的和是否与 `[rsp + 0x8]` 相同（即第二个输入的数字），如果都相同则通过，反之引爆炸弹。

输入内容获取：

①由于寻址乘数的系数为 `int` 类型的长度 4，且基于同一个地址，故该地址对应的位置应为一个数组，由于数字范围为 0 – 15，共 16 个数字，故数组大小应为 16 使用指令【`x/16dw 0x401ba0`】进行对应数组内容的查看。

②如图 19 所示，即为该地址下所对应的数组。故有表二所示对应关系。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```
(gdb) x/16dw 0x401ba0
0x401ba0 <array.3014>: 10      2      14      7
0x401bb0 <array.3014+16>: 8      12     15     11
0x401bc0 <array.3014+32>: 0      4      1      13
0x401bd0 <array.3014+48>: 3      9      6      5
```

图 19 0x401ba0 处的数组

编号	0	1	2	3
数组内容	10	2	14	7
编号	4	5	6	7
数组内容	8	12	15	11
编号	8	9	10	11
数组内容	0	4	1	13
编号	12	13	14	15
数组内容	3	9	6	5

表 2 0x401ba0 处的数组与其编号

③由表 2 与代码运行逻辑中③的逻辑内容，我们可以获取到一个寻址的循环：

$$5 \rightarrow 12 \rightarrow 3 \rightarrow 7 \rightarrow 11 \rightarrow 13 \rightarrow 9 \rightarrow 4 \downarrow$$
$$\uparrow 15 \leftarrow 6 \leftarrow 14 \leftarrow 2 \leftarrow 1 \leftarrow 10 \leftarrow 0 \leftarrow 8$$

表 3 数组循环

④由表 3 与代码逻辑中④的内容，根据数组循环逆推 15 前第 12 位数字，即 7，数组循环可知中间经过的数字为 11、13、9、4、8、0、10、1、2、14、6、15，总和为 93，故最终的答案为【7 93】，如图 20，输入答案【7 93】，可以看到对应提示，即答案正确。

```
So you got that one. Try this one.
7 93
Congratulations! You've (mostly) defused the bomb!
Hit Control-C to escape phase 6 (for free!), but if you want to
try phase 6 for extra credit, you can continue. Just beware!
```

图 20 gdb 测试第四部分对应答案①

⑤由于函数本身一开始对于输入的第一个数字作了截断，故其实答案可以为【 $k * 32 + 7 93$ 】，其中 k 为任意整数。其中 k 取 1 时有答案【39 93】、 k 取 -1 时有答案【-25 93】，如图 21、22 所示，输入后也可看到对应提示，说明答案正确。

```
So you got that one. Try this one.
39 93
Congratulations! You've (mostly) defused the bomb!
Hit Control-C to escape phase 6 (for free!), but if you want to
try phase 6 for extra credit, you can continue. Just beware!
```

图 21 gdb 测试第四部分对应答案②

```
So you got that one. Try this one.
-25 93
Congratulations! You've (mostly) defused the bomb!
Hit Control-C to escape phase 6 (for free!), but if you want to
try phase 6 for extra credit, you can continue. Just beware!
```

图 22 gdb 测试第四部分对应答案③

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

7. phase_6|关卡⑥

如图 23 所示，即为 phase_6 所对应代码：

```
553 0000000004010d9 <phase_6>:
554 4010d9: 48 83 ec 08      sub    $0x8,%rsp
555
556 4010dd: ba 0a 00 00 00   mov    $0xa,%edx # edx = 0xa
557 4010e2: be 00 00 00 00   mov    $0x0,%esi # esi = 0x0
558 4010e7: e8 94 fa ff ff   call   400b80 <strtol@plt>
559 4010ec: 89 05 8e 16 20 00 mov    %eax,0x20168e(%rip) # (0x602780) # node0 = eax
560 # 602780 <node0>
561 4010f2: bf 80 27 60 00   mov    $0x602780,%edi # edi = 0x602780
562 # 节点初始化
563 4010f7: e8 73 ff ff ff   call   40106f <fun6>
564 4010fc: 48 8b 40 08      mov    0x8(%rax),%rax # rax = *(rax + 0x8), 第一个链表
565 401100: 48 8b 40 08      mov    0x8(%rax),%rax # rax = *(rax + 0x8), 第二个链表
566 401104: 48 8b 40 08      mov    0x8(%rax),%rax # rax = *(rax + 0x8), 第三个链表
567 401108: 8b 15 72 16 20 00 mov    0x201672(%rip),%edx # edx = *(0x201672),第四个链表 # 602780 <node0>
568 40110e: 39 10            cmp    %edx,%rax
569 401110: 74 05            je     401117 <phase_6+0x3e> # if edx == *(rax) goto end
570 401112: e8 26 05 00 00   call   40163d <explode_bomb>
571
572 401117: 48 83 c4 08      add    $0x8,%rsp
573 40111b: c3              ret
```

图 23 汇编代码中 phase_6 函数

代码主要逻辑分析：

- ①除去初始化，则首先对节点进行所对应的初始化的寄存器初始化，然后初始化节点，并传入函数输入的数字[%eax]对节点进行赋值。且链表头位于 0x602780。
- ②将节点传入<fun6>中进行排序，<fun6>的代码如图 24 所示：

```
506
507 00000000040106f <fun6>:
508 40106f: 4c 8b 47 08      mov    0x8(%rdi),%r8 # r8 = *(rdi + 0x8)
509 401073: 48 c7 47 08 00 00 00 movq    $0x0,0x8(%rdi) # *(rdi + 0x8) = 0x0
510 40107a: 00
511 40107b: 48 89 f8          mov    %rdi,%rax # rax = rdi
512 40107e: 48 89 f9          mov    %rdi,%rcx # rcx = rdi
513 401081: 4d 85 c0          test   %r8,%r8
514 401084: 75 40            jne    4010c6 <fun6+0x57> # if r8 <= 0 goto 4010c6
515 401086: 48 89 f8          mov    %rdi,%rax # rax = rdi
516 401089: c3              ret
517
518 40108a: 48 89 d1          mov    %rdx,%rcx # rcx = rdx
519 40108d: 48 8b 51 08      mov    0x8(%rcx),%rdx # rdx = *(rcx + 0x8)
520 401091: 48 85 d2          test   %rdx,%rdx
521 401094: 74 09            je     40109f <fun6+0x30> # if rdx == 0 goto 40109f
522 401096: 39 32            cmp    %esi,(%rdx)
523 401098: 7f f0            jg     40108a <fun6+0x1b> # if esi > *(rdx) goto 40108a
524 40109a: 48 89 cf          mov    %rcx,%rdi # rdi = rcx
525 40109d: eb 03            jmp    4010a2 <fun6+0x33> # goto 4010a2
526 # 相等与后方进行排序
527 40109f: 48 89 cf          mov    %rcx,%rdi # rdi = rcx
528
529 4010a2: 48 39 d7          cmp    %rdx,%rdi
530 4010a5: 74 06            je     4010ad <fun6+0x3e> # if rdx == rdi goto 4010ad
531 4010a7: 4c 89 47 08      mov    %r8,0x8(%rdi) # *(rdi + 0x8) = r8
532 4010ab: eb 03            jmp    4010b0 <fun6+0x41> # goto 4010b0
533 # 向前寻找
534 4010ad: 4c 89 c0          mov    %r8,%rax # rax = r8
535
536 4010b0: 49 8b 48 08      mov    0x8(%r8),%rcx # rcx = *(r8 + 0x8)
537 4010b4: 49 8b 50 08      mov    %rdx,0x8(%r8) # *(r8 + 0x8) = rdx
538 4010b8: 48 85 c9          test   %rcx,%rcx
539 4010bb: 74 1a            je     4010d7 <fun6+0x68> # if rcx == 0 goto end
540 4010bd: 49 89 c8          mov    %rcx,%r8 # r8 = rcx
541 4010c0: 48 89 c1          mov    %rax,%rcx # rcx = rax
542 4010c3: 48 89 c7          mov    %rax,%rdi # rdi = rax
543 # 向后寻找
544 4010c6: 48 89 ca          mov    %rcx,%rdx # rdx = rcx
545 4010c9: 48 85 c9          test   %rcx,%rcx
546 4010cc: 74 d4            je     4010a2 <fun6+0x33> # if rcx == 0 goto 4010a2
547 4010ce: 41 8b 30          mov    (%r8),%esi # esi = *(r8)
548 4010d1: 39 31            cmp    %esi,(%rcx)
549 4010d3: 7f b8            jg     40108d <fun6+0x1e> # if esi > *(rcx) goto 40108d
550 4010d5: eb cb            jmp    4010a2 <fun6+0x33> # goto 4010a2
551 # 返回升序链表序列
552 4010d7: f3 c3            repz ret
```

图 24 汇编代码中 fun6 函数

由此可得 fun6 对链表进行升序排序。

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

③逐个进行链表的递归，获取链表中第四个链表中存储的数字，并将其与一开始的数字进行比较，若相同则结束，反之引爆炸弹（即链表中第四个节点需为一开始创建的新节点）。

输入内容获取：

①如图 25 所示，输入指令【x/40dw 0x602780】查看链表内容

```
(gdb) x/40dw 0x602780
0x602780 <node0>:      0      0      6301584 0
0x602790 <node1>:     374     1      6301600 0
0x6027a0 <node2>:     826     2      6301616 0
0x6027b0 <node3>:     370     3      6301632 0
0x6027c0 <node4>:     782     4      6301648 0
0x6027d0 <node5>:     488     5      6301664 0
0x6027e0 <node6>:     673     6      6301680 0
0x6027f0 <node7>:     286     7      6301696 0
0x602800 <node8>:     600     8      6301712 0
0x602810 <node9>:     529     9         0    0
(gdb)
```

图 25 0x602780 中链表内容

可以看出链表中第一项为存储的数据，第二项为序列，而第三项为指向的下一个节点，最后一个节点为0x0，为空指针，符合链表的结构。

②对图 23 中的链表数字进行排序，可以知道第三和第四大的数字分别为673和600，故输入的数字所产生的节点的大小要在600 – 673间，取数字【630】作为答案进行输入，如图 24，可以看到对应提示，即答案正确，完成实验。

```
Hit Control-C to escape phase 6 (for free!), but if you want to
try phase 6 for extra credit, you can continue. Just beware!
630
Congratulations! You've defused the bomb! Again!
```

图 26 gdb 测试第五部分对应答案

四、实验结论

①本次逆向工程实验通过动态调试（GDB）与静态反汇编（objdump）相结合的方式，深入剖析了程序底层运行机制及核心逻辑。实验过程中，成功解析了多阶段程序验证逻辑并成功拆除了所有的炸弹。

②其中，在第三阶段中，通过分析 0x401ba0 处数组编号与循环寻址关系，逆推出关键路径序列并计算总和，验证了“7 93”及其扩展形式（如 39 93、-25 93）作为有效输入的正确性；

③在第六阶段中，结合链表结构（头地址 0x602780）与 fun6 排序函数的行为，确定需构造数值范围位于 600-673 之间的节点以满足第四个节点匹配条件，最终以输入“630”完成验证。

④在实验过程中，我不仅掌握了寄存器状态监控、内存数据解析、函数调用栈跟踪等调试技巧，更深刻理解了程序控制流保护机制、数据结构动态操作及整数溢出处理等底层特性。更彻底的理解并应用了汇编代码的作用。

⑤最后，通过理论推导与实践验证的闭环过程，验证了逆向分析对程序行为预测的有效性，成功完成实验。

⑥除此之外,需要说明的是，题目当中 scanf 的输入部分是来源于获取的地址的，这部分由于在题目中没有改动便没有提到。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

指导教师批阅意见:

成绩评定:

指导教师签字：马晨琳

2024 年 5 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。